

Java Code Structure & Readability

Project Structure

Organize your project into packages based on functionality:
src/

- ■ ■ ■ com.example.project/
- ■ ■ ■ Main.java
- ■ ■ ■ model/ # Data classes (POJOs)
- ■ ■ ■ service/ # Business logic
- ■ ■ ■ controller/ # Input/output handling
- ■ ■ ■ repository/ # Data access layer
- ■ ■ ■ util/ # Utility/helper classes

Class Structure

Follow a consistent order within each class:

```
package com.example.project.model;

import java.util.List; // 1. Imports

public class User {
// 2. Class declaration

// 3. Constants
public static final int MAX_AGE = 120;

// 4. Fields
private String name;
private int age;

// 5. Constructors
public User(String name, int age) {
this.name = name;
this.age = age;
}

// 6. Getters and Setters
public String getName() { return name; }
public void setName(String name) { this.name = name; }

// 7. Business Methods
public boolean isAdult() {
return age >= 18;
}

// 8. toString(), equals(), hashCode()
@Override
public String toString() {
return "User{name=\"" + name + "\", age=\"" + age + "\"}";
}
```

```
}  
}
```

Readability Best Practices

Naming Conventions

- Classes: PascalCase → UserAccount
- Methods/Variables: camelCase → calculateTotal()
- Constants: UPPER_SNAKE_CASE → MAX_USERS

Code Formatting

- Use consistent indentation (4 spaces is standard).
- Add blank lines between logical sections.
- Keep lines under 100 characters.

Comments & Documentation

- Use Javadoc for public classes and methods:

```
/**  
 * Calculates the total price including tax.  
 * @param price Base price  
 * @param taxRate Tax rate as a decimal  
 * @return Total price  
 */  
public double calculateTotal(double price, double taxRate) { ... }
```

- Use inline comments sparingly and only when necessary.

Avoid

- Long methods (split into smaller ones).
- Deep nesting (use guard clauses).
- Magic numbers (use constants).